



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

Deep Learning Application

Reinforcement Learning

Edoardo Sarri

7173337

Giugno 2025

Indice

1	Introduzione	4
1.1	Main	4
1.2	Moduli Python	4
2	Analisi	5
2.1	Cart Pole	5
2.1.1	Descrizione	5
2.1.2	Obiettivo	5
2.2	Lunar Lander	5
2.2.1	Descrizione	6
2.2.2	Obiettivo	6
3	Cart Pole	7
3.1	Baseline	7
3.1.1	Nessuna baseline	7
3.1.2	Standardizzazione	8
3.1.3	Baseline con funzione di valore	8
3.2	Iperparametri	10
4	Lunar Lander	12
4.1	Baseline	12
4.2	Architettura rete neurale	13
4.3	Step scheduler	13
4.4	Gradient clipping	13

Elenco delle figure

3.1	Esperimenti senza baseline.	7
3.2	Esperimenti con standardizzazione.	8
3.3	Ricompense con baseline state-value function.	9
3.4	Confronto tra le tecniche.	9
3.5	Loss della policy con baseline state-value function.	9
3.6	Learning rate circa $1e-5$	10
3.7	Sweep su γ e learning rate.	11
3.8	Confronti tra esperimenti.	11
4.2	Varie architetture.	13
4.3	Scheduler StepLR.	14
4.4	Risultati con gradient clipping.	14

1 Introduzione

In questo primo capitolo viene spiegata l'organizzazione del progetto.

1.1 Main

Il main del progetto è rappresentato dal file `main.py`. Inizialmente era stato valutato l'uso di un Jupyter notebook, ma il kernel di Jupyter non funziona bene con gli ambienti di Gymnasium, in particolare con quelli che usano ambienti grafici come Pygame. L'addestramento dell'agente procedeva senza troppi problemi, ma quando si andava a visualizzare l'agente, Pygame andava in crash.

Questo documento è pensato per non far eseguire il main a chi è interessato solo ai risultati. Ogni risultato loggato su W&B [4] (framework usato durante tutto il progetto per tracciare gli esperimenti) o esperimento eseguito è riassunto in questa relazione. Con questa idea i vari file `.py` sono utili non per capire cosa è stato fatto, ma come è stato fatto.

1.2 Moduli Python

Le funzioni chiamate in `main.py` sono importate da più moduli:

- `reinforce.py`
Contiene la funzione che implementa l'algoritmo REINFORCE.
- `myModel.py`
Contiene le reti neurali usate nel processo di addestramento dell'agente.
- `core.py`
Contiene le funzioni fondamentali richiamate dall'algoritmo REINFORCE
- `sweep.py`
Contiene il codice usato per implementare lo sweep per ricercare i migliori iperparametri in Cart Pole.
- `utility.py`
Contiene le funzioni ausiliarie utili nel progetto.

2 Analisi

Gli ambienti in cui si muove l'agente sono definiti in Gymnasium [2], una libreria Python open source ideata per sviluppare algoritmi di Reinforcement learning.

2.1 Cart Pole

Il primo ambiente utilizzato è stato Cart Pole [1]. L'obiettivo dell'agente in questo contesto è quello di tenere in equilibrio una barra posta su un carrello e soggetta alla forza di gravità.

Ci sono due versioni di Cart Pole. Quella utilizzata in questo progetto è CartPole-v1.

2.1.1 Descrizione

L'action space dell'ambiente è composto da due azioni: muovere il carrello verso sinistra è rappresentato con uno zero; muoverlo verso destra con un 1.

L'observation space è definito da quattro variabili: la posizione e la velocità del carrello, l'angolo e la velocità angolare della barra.

Per ogni azione eseguita all'interno dell'episodio, l'agente ottiene una reward di +1.

2.1.2 Obiettivo

L'episodio termina quando la barra supera i $\pm 12^\circ$ di inclinazione o se la posizione del carrello supera un range di $\pm 2,4$. Inoltre l'episodio finisce in maniera automatica quando vengono superate le 500 iterazioni.

L'obiettivo è raggiungere una reward stabile di almeno 475 punti per episodio.

2.2 Lunar Lander

Il secondo ambiente utilizzato è stato Lunar Lander [3]. L'agente in questo caso è una navicella, dotata di motore principale e laterale, che deve atterrare in un preciso spazio nell'ambiente.

Ci sono due versioni di Lunar Lander, una discreta e una continua: nella prima i motori sono sempre accesi alla massima potenza; nella seconda abbiamo un valore che ci indica la potenza emessa. La versione utilizzata in questo progetto è quella discreta.

2.2.1 Descrizione

L'action space dell'ambiente è composto da quattro azioni: non fare niente, attivare il motore centrale, quello destro o quello sinistro.

L'observation space è definito da otto variabili: le coordinate della navicella in due dimensioni, la sua velocità lineare in due dimensioni, il suo angolo, la velocità angolare e due booleani che indicano l'appoggio di una gamba sul terreno.

Le reward possono diminuire e aumentare all'interno di un episodio a seconda se l'agente è più o meno vicino rispetto alla zona di atterraggio, alla velocità dell'agente e alla sua inclinazione. Inoltre ottiene 10 punti per ogni gamba che poggia a terra, diminuisce di 0,03 punti per ogni frame dove è in azione un motore laterale e di 0,3 per ogni frame in cui è in azione quello principale; riceve ancora ± 100 punti per un atterraggio o un crash.

È interessante notare che quando si importa l'ambiente si possono settare la velocità del vento, la forza di gravità e il rumore sulla potenza dei motori.

2.2.2 Obiettivo

L'episodio termina quando la navicella ha un crash o se esce dai limiti del box che la contiene.

L'obiettivo è raggiungere una reward stabile di almeno 200 punti per episodio.

3 Cart Pole

In questo capitolo cerchiamo di risolvere l'ambiente Cart Pole, introdotto nella Sezione 2.1.

Per verificare il raggiungimento dell'obiettivo, abbiamo considerato un ritorno medio per episodio di almeno 475 punti. Il comportamento dell'agente viene osservato tramite un grafico che mostra la somma delle ricompense per epoca; a causa del forte rumore, il grafico è stato rappresentato utilizzando una media mobile esponenziale (Time Weighted EMA) con fattore 0,95. Questa scelta consente di generalizzare anche ad altri ambienti, come vedremo nel prossimo capitolo.

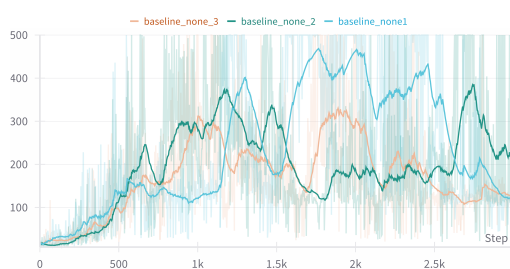
3.1 Baseline

In questa sezione analizziamo diverse strategie di baseline. Siamo partiti da un'implementazione di REINFORCE senza baseline, proseguendo con una standardizzazione dei returns e infine con una stima della funzione di valore dello stato.

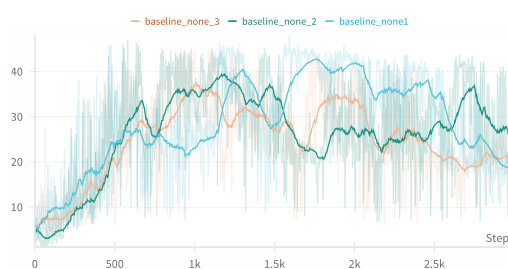
3.1.1 Nessuna baseline

I primi esperimenti sono stati condotti senza alcuna baseline.

Come si osserva nella Figura 3.1a, senza baseline l'apprendimento risulta molto irregolare e, anche dopo 3000 episodi, l'ambiente non viene risolto in modo stabile; solo il primo esperimento si avvicina a una possibile soluzione. Le tre iterazioni mostrano risultati piuttosto differenti.



(a) Somma delle ricompense



(b) Loss

Figura 3.1: Esperimenti senza baseline.

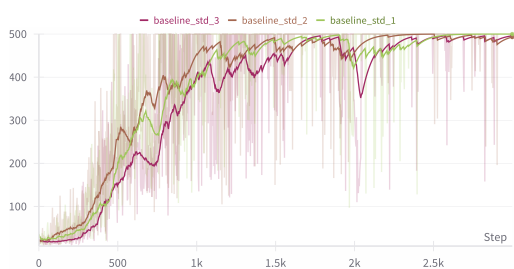
Un'osservazione interessante riguarda la loss: come mostrato in Figura 3.1b, questa è altamente irregolare e non mostra un andamento decrescente tipico dell'apprendi-

mento supervisionato. Ciò deriva dalla natura stocastica dell'algoritmo: i returns variano a ogni episodio e influenzano direttamente la loss, che è anch'essa funzione della policy, aggiornata a ogni episodio.

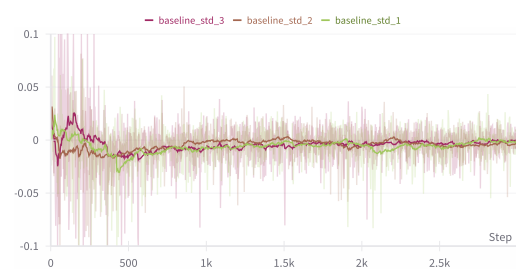
3.1.2 Standardizzazione

Per migliorare la stabilità dell'apprendimento, abbiamo standardizzato i returns all'interno di ogni episodio.

Come mostrato in Figura 3.2a, questa tecnica produce un apprendimento più regolare. L'ambiente viene risolto intorno ai 1500 episodi, con somme delle ricompense costantemente superiori a 475.



(a) Ricompense standardizzate



(b) Loss

Figura 3.2: Esperimenti con standardizzazione.

Osservando la loss in Figura 3.2b, notiamo che i suoi valori sono contenuti in un intervallo ristretto attorno a ± 1 , effetto diretto della standardizzazione dei returns.

3.1.3 Baseline con funzione di valore

In questo caso, la baseline è rappresentata dalla funzione di valore dello stato, approssimata da una rete neurale con la stessa architettura di quella della policy.

Questa tecnica rende gli esperimenti più prevedibili, con risultati meno variabili tra le esecuzioni, come mostrato in Figura 3.3.

Il confronto con la standardizzazione (Figura 3.4a) mostra che questa tecnica offre maggiore stabilità e una migliore velocità di convergenza.

Dal punto di vista computazionale (Figura 3.4b), l'implementazione senza baseline è la più veloce, mentre l'approssimazione della funzione di valore è la più lenta.

Analizzando la loss della policy (Figura 3.5), si nota un andamento inizialmente simile al caso senza baseline, ma che con il progredire degli episodi tende a convergere.

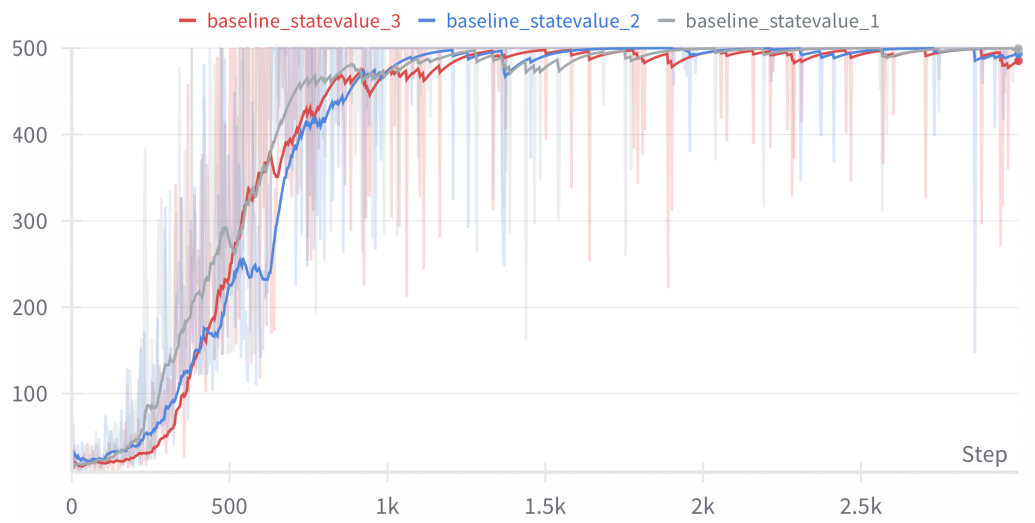


Figura 3.3: Ricompense con baseline state-value function.



(a) Confronto ricompense

(b) Tempi di esecuzione

Figura 3.4: Confronto tra le tecniche.

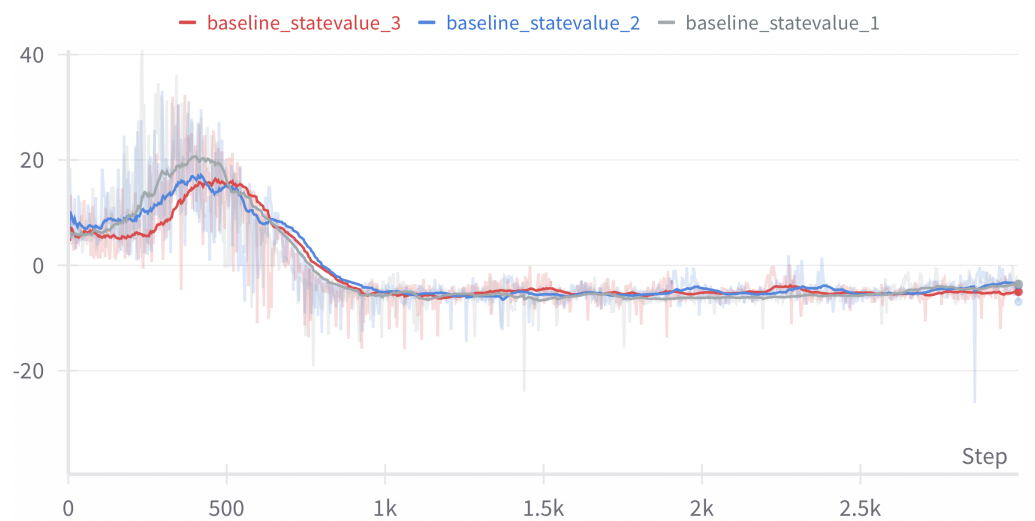


Figura 3.5: Loss della policy con baseline state-value function.

3.2 Iperparametri

Finora, il fattore di sconto γ e il learning rate sono stati impostati rispettivamente a 0,99 e $1e-3$. In questa sezione analizziamo l'effetto della loro variazione. I valori esplorati sono: γ tra 0,9 e 0,999 e learning rate tra $1e-5$ e $1e-2$. La baseline utilizzata è l'approssimazione della funzione di valore.

Il grafico principale di riferimento resta quello della somma dei returns per epoca, filtrato con una Time Weighted EMA con fattore 0,99.

La Figura 3.6 mostra tre esperimenti con learning rate molto basso (circa $1e-5$) e γ tra 0,93 e 0,97. L'agente non riesce ad apprendere.

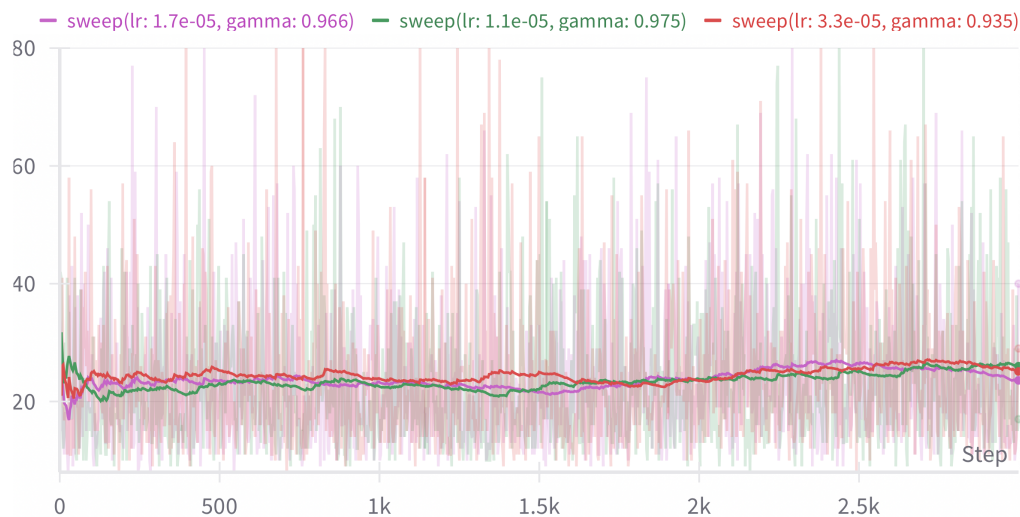


Figura 3.6: Learning rate circa $1e-5$.

La Figura 3.7 raccoglie altri esperimenti con combinazioni diverse. I migliori risultati si ottengono con γ elevati, confermando la sensibilità dell'algoritmo REINFORCE a questi iperparametri.

In Figura 3.8a, osserviamo che valori bassi di γ impediscono la risoluzione dell'ambiente. Al contrario, come mostrato in Figura 3.8b, un γ elevato migliora l'apprendimento. Learning rate più alti portano a una rapida risoluzione, ma con peggioramento successivo; learning rate più bassi rendono l'apprendimento più lento ma stabile.

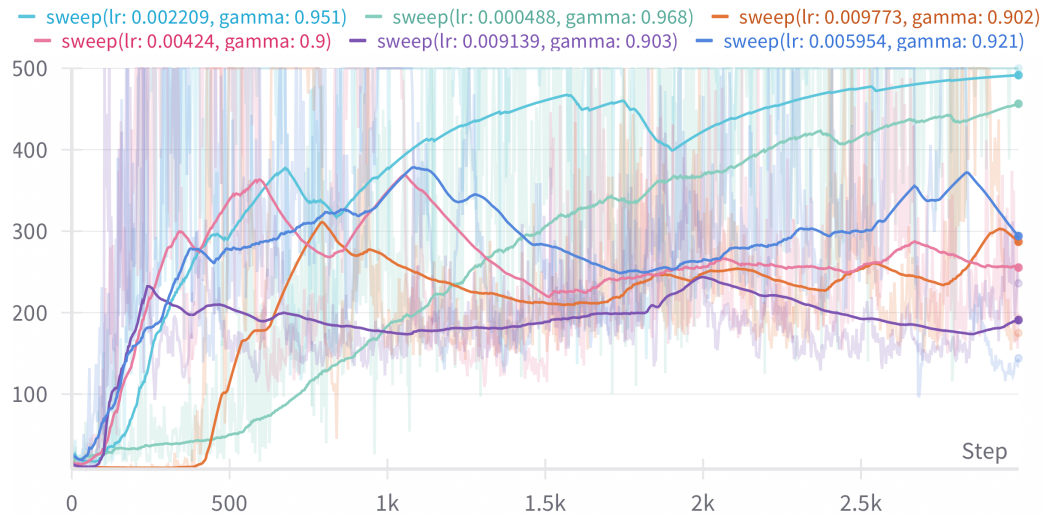
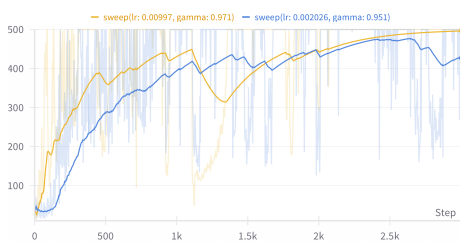
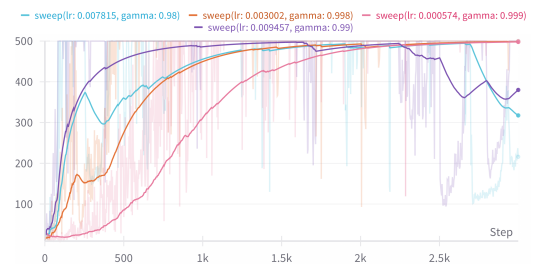


Figura 3.7: Sweep su γ e learning rate.



(a) γ bassi



(b) γ alti

Figura 3.8: Confronti tra esperimenti.

4 Lunar Lander

In questo capitolo cercheremo di risolvere l'ambiente Lunar Lander analizzato nella Sezione 2.2.

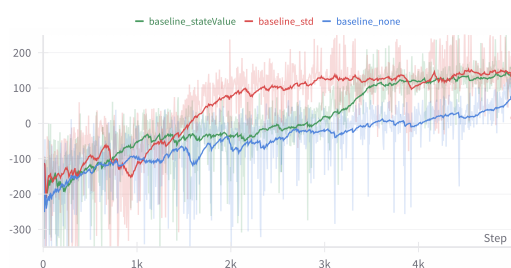
Come criterio per verificare l'obiettivo si è considerato il raggiungimento in modo consistente di una returns per episodio di 200 punti. Il grafico che ci farà osservare questo comportamento è quello che mette in evidenza la somma delle reward per ogni epoca; vista l'elevata rumorosità, è stato rappresentato con un Time Weighted EMA con un fattore di 0,95.

4.1 Baseline

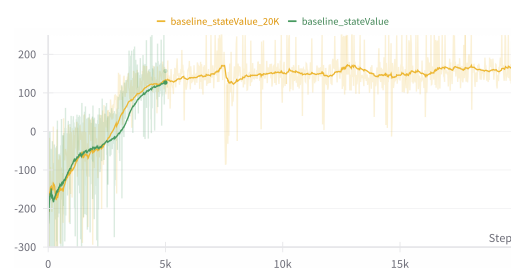
Per cercare di risolvere l'ambiente utilizzando gli stessi metodi di Cart Pole, sono stati eseguiti tre esperimenti usando le tre baseline definite nel Capitolo 3: senza baseline, con la standardizzazione e con l'approssimazione della state-value function tramite una rete neurale. Siccome l'ambiente è più complesso rispetto a Cart Pole, ho deciso di addestrare l'agente per 5000 episodi, contro le 3000 usate in precedenza.

Come si può osservare dai risultati in Figura 4.1a, mentre risolvere Cart Pole era banale, adesso il compito è più complesso. Si può notare che la soluzione senza baseline fa molta fatica, e in praticamente nessun episodio è stata raggiunta una somma dei rewards superiore a 200. Con la standardizzazione e la state-value function la situazione migliora: ci sono degli episodi in cui l'ambiente è risolto, ma questo succede in modo non stabile e quindi il risultato non ci soddisfa. Usare più episodi (20K) non migliora l'addestramento, come si vede in Figura 4.1b; dobbiamo quindi cercare di cambiare un po' le cose per cercare una soluzione più soddisfacente.

Come osservato nel capitolo precedente possiamo notare come la soluzione con la baseline che approssima la state-value function è più stabile.



(a) Tre baseline.



(b) Baseline con state-value function.

4.2 Architettura rete neurale

Nel seguito i vari esperimenti sono stati eseguiti con la state-value function baseline. In questo capitolo vediamo come è stata cambiata la rete che approssima la policy e la state-value function; ricordiamo che queste due hanno la stessa architettura.

Sono state provate varie architetture: la profondità è di tre layer interni tutti con lo stesso numero di neuroni; in un caso 32, in un altro 64 e nell'ultimo 128. Visto il contesto molto stocastico, sono stati fatti tre esperimenti per ogni architettura, ognuno dei quali è stato eseguito per 10000 episodi.

Osserviamo i vari esperimenti fatti. Come si può notare dalle Figure 4.2, all'aumentare della complessità della rete aumenta anche la differenza di risultati da un esperimento all'altro. Si può anche osservare che una rete più complessa sembra essere più promettente rispetto a una più semplice nell'arrivare al risultato, cioè ai 200 punti per episodio.

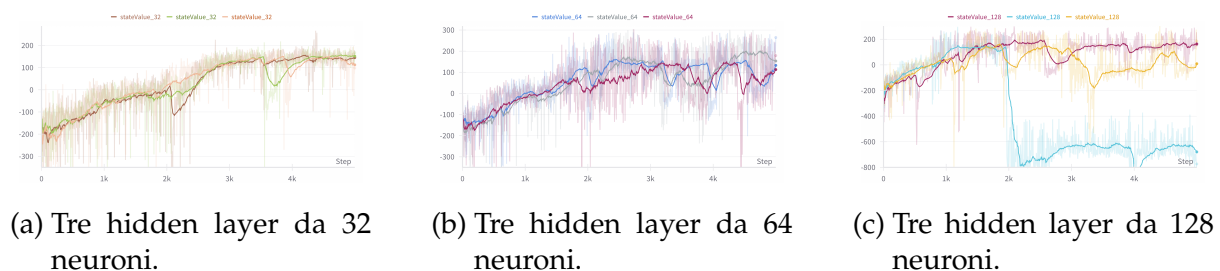


Figura 4.2: Varie architetture.

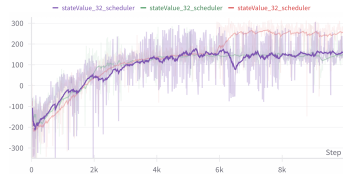
4.3 Step scheduler

Per cercare di migliorare la stabilità dell'addestramento è stato introdotto uno scheduler per i due ottimizzatori delle reti che approssimano la policy e la state-value function. Lo scheduler in questione è uno dei più semplici, cioè StepLR: ogni 1500 episodi il learning rate è decrementato di un fattore di 0,9.

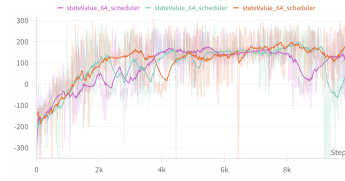
I risultati ottenuti sono quelli mostrati in Figura 4.3. Come si può osservare, la stabilità è migliorata leggermente, soprattutto per il modello più complesso. Nonostante questo, i risultati non sono del tutto soddisfacenti per quanto riguarda le performance.

4.4 Gradient clipping

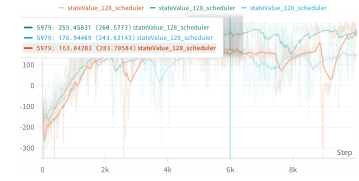
Il metodo che forse ha dato il miglioramento più evidente è stato quello di limitare la norma dei gradienti per evitare il fenomeno dell'exploding gradient, molto comune nel reinforcement learning.



(a) Tre hidden layer da 32 neuroni.



(b) Tre hidden layer da 64 neuroni.



(c) Tre hidden layer da 128 neuroni.

Figura 4.3: Scheduler StepLR.

Sono stati testati tre valori per il massimo delle norme: 2, 5 e 10. Come si può notare dalla Figura 4.4, i risultati migliori li abbiamo quando cerchiamo di limitare maggiormente la norma dei gradienti; questo ci fa capire quanto sia importante il fenomeno dell'exploding gradient in questo contesto.

Con questo risultato si può dire, anche osservando la visualizzazione, che l'ambiente è risolto.

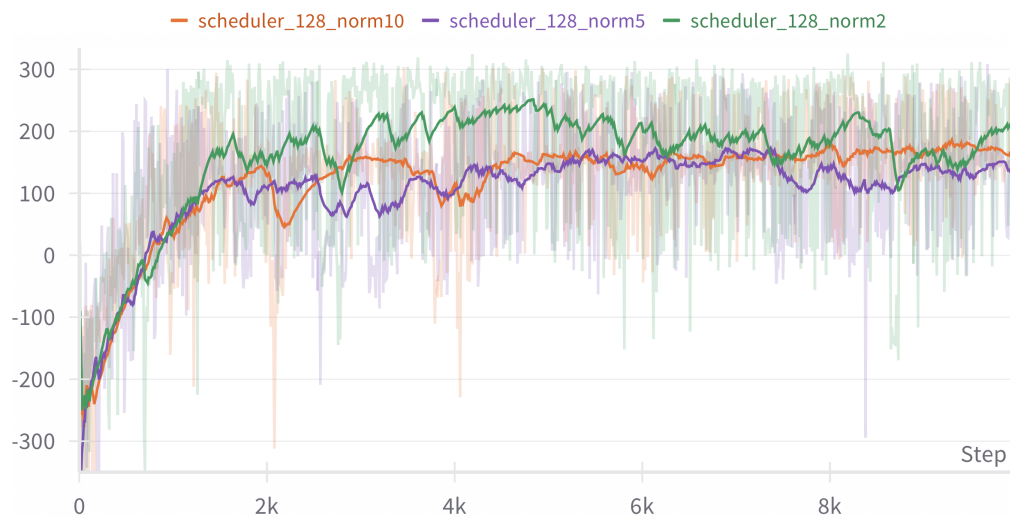


Figura 4.4: Risultati con gradient clipping.

Bibliografia

- [1] *Cart Pole*. https://www.gymlibrary.dev/environments/classic_control/cart_pole/.
- [2] *Gym*. <https://www.gymlibrary.dev/index.html>.
- [3] *Lunar Lander*. https://gymnasium.farama.org/environments/box2d/lunar_lander/.
- [4] *Weight & Biases*. <https://wandb.ai/site/>.